Learning Control Policies from Expert Videos

Adrian Kazi akazi35@gatech.edu
Matteo Bigliardi mbigliardi3@gatech.edu
Nuwan Werellagama nwerellagama3@gatech.edu
Sebastian Nickels snickels3@gatech.edu

Abstract

Video demonstrations can provide useful information about how tasks are performed, but they usually do not contain the actions, rewards, or internal states that produced them. In this work, we study imitation from observation in `LunarLanderContinuous-v3`*, where a student policy attempts to recover useful control behavior from expert videos. We compare four approaches along a spectrum from reinforcement-learning-based reward construction, through supervised and action-free imitation, toward world-model learning from video.*

Our results suggest that RL-based methods provide useful and interpretable baselines, while more general video-based methods remain limited by action grounding, controllability, and error accumulation. This highlights the central difficulty of video-to-policy learning: visual demonstrations are scalable, but converting them into reliable closed-loop control remains unresolved.

1. Introduction

In this project, we studied whether an agent can learn useful behavior from video before that behavior is translated into actions for a specific control system. In many domains, it is easier to record skilled behavior than to obtain the exact actions, rewards, or internal states that produced it. Our test case is `LunarLanderContinuous-v3`: we trained a teacher agent, recorded its expert rollouts as video, and investigated whether a student agent could recover useful control behavior from those visual demonstrations.

Recent progress in large-scale pretraining has shown that useful capabilities can emerge from training on large amounts of relatively cheap data. Video is a natural next step: it is abundant, inexpensive to collect, and contains information about how objects, agents, and environments change over time. If agents can learn from video alone, then existing visual demonstrations could become training data for new control systems, reducing the need for manually labeled actions, hand-designed rewards, or carefully instrumented environments.

To study this problem, we implemented four complementary approaches. They are organized along a spectrum from methods that remain close to reinforcement learning and explicit control supervision to methods that rely more heavily on learned visual representations and forward prediction.

- **Discriminator-Based Latent State Transitions** (`nw-dev`): an RL-based formulation that uses a discriminator to transform expert-like latent transitions into a reward signal for policy optimization. This was the most directly RL-grounded approach and produced the strongest policy-level results in our experiments.
- **Supervised Inverse-Dynamics and Behavioral Cloning** (`mb-dev`): a control-supervised imitation baseline that learns actions from labeled visual transitions and then trains a behavioral-cloning policy.
- **Behavioral Cloning from Observation (BCO)** (`sn-dev`): an action-free extension of the inverse-dynamics baseline. The inverse-dynamics model is trained on random exploration data, then used to pseudo-label expert videos with inferred actions.
- **World Model + Machine Forward** (`ak-dev`): a deep-learning-based formulation that learns latent dynamics from unlabeled video through forward prediction. This is the most general direction in principle, but remains the least resolved because action grounding and controllability are still open problems.

Overall, the project follows a progression from RL-based reward learning, through supervised and action-free imitation, toward deep-learning-based world-model generalization.

Standard reinforcement learning learns by interacting with an environment and optimizing a reward signal, but this can require many samples and carefully designed rewards. Standard imitation learning, such as behavioral cloning, is often simpler, but usually assumes access to expert actions. Our setting is closer to imitation from observation, where expert behavior is available primarily as visual change over time. This connects our work to Behavioral Cloning from Observation, which uses inverse dynamics to

infer actions from observation transitions, and to Video Pre-Training, which shows that video can provide a useful behavioral learning signal when grounded by action-labeled data [13, 15, 1].

Our dataset was self-generated in the `LunarLanderContinuous-v3` Gymnasium environment [16]. Expert rollouts were produced by a trained TD3 teacher policy, recorded as videos, and converted into ordered frame sequences. Since the focus of this report is exclusively on the student models, we do not describe the teacher architecture in detail here. Further details about the teacher implementation are available in the referenced repository and its README [8].

Frames were converted to grayscale, resized to 84×84, normalized, and encoded into latent vectors by the autoencoder. Depending on the experimental direction, the same demonstrations were represented as raw frame pairs, latent transition pairs, latent sequences, or aligned transition files containing frames, states, actions, rewards, and terminal flags. Generating the data ourselves made the experiments reproducible and gave us control over rendering, frame order, and evaluation conditions. Following dataset documentation recommendations [2], the self-generated dataset gave us control over rendering, frame order, teacher policy, and transition alignment for reproducible comparison.

2. Approach

We used our shared project repository [8]. The TD3 teacher was adapted from previous work by the project coauthors and modified for this project to generate expert rollouts, record videos, align frames with states and actions, and provide data for the student-side imitation experiments.

Starting from this, we implemented a reproducible `video-to-policy` pipeline in which expert behavior is generated in `LunarLanderContinuous-v3`, recorded as videos, and converted into visual transition data. The common first step is a convolutional autoencoder, which maps each rendered frame o_t into a compact latent representation $z_t = Encoder(o_t)$. This latent space is used across the student-side experiments, so that each method works with learned visual features instead of raw images.

Before training sequence-based models, we reduced temporal redundancy in the video data. Consecutive frames in expert rollouts are often nearly identical, so we used a frame-selection function that keeps a new frame only when it contributes enough new visual signal relative to the previous kept frame. This produced shorter but more informative sequences with stronger apparent motion. We also used a sequence-formatting step to preserve the start and end of each trajectory while selecting a fixed number of frames, so that each episode could be represented as a tensor suitable for sequential processing.

The first direction was **Discriminator-Based Latent State Transitions**. Inspired by the Generative Adversarial Imitation from Observation (*GAIfO*) pipeline by Torabi et al. [14], expert videos were encoded into latent transitions (z_t, z_{t+1}), while learner rollouts produced transitions in the same latent space. A discriminator was trained to distinguish expert transitions from learner transitions. Its output was then used as a synthetic reward for PPO, replacing the true environment reward during student training. The goal was to make learner transitions indistinguishable from expert transitions. Our full pipeline implementation is outlined in Figure 1.

The second direction was **Supervised Inverse-Dynamics and Behavioral Cloning**. Given latent transition features $[z_t, z_{t+1}, z_{t+1} - z_t]$, we trained an inverse-dynamics model to infer actions from labeled expert transitions. We then trained a behavioral-cloning policy $z_t \rightarrow a_t$. This tested whether actions were recoverable from visual latent transitions, and whether those inferred actions were enough to train a student policy.

The third direction was **Behavioral Cloning from Observation**. This approach kept the same inverse-dynamics idea, but removed direct dependence on labeled expert actions. The inverse-dynamics model was trained on random exploration data instead of labeled expert transitions. It was then used to pseudo-label expert videos, and those inferred actions were used to train a behavioral-cloning policy. This tested whether action-free expert videos could still be converted into useful policy supervision.

The fourth direction was **World Model + Machine Forward**. Instead of directly inferring actions, this approach learned latent dynamics from video and predicted future latent states through forward modeling. This was harder to optimize, but it tested the most general version of the problem: whether useful behavior can come from learned visual dynamics rather than explicit rewards or action labels.

Across these experiments, our strategy was to compress videos into latent states and test several ways to turn visual change into actions. The approach does not claim a new PPO, discriminator, autoencoder, or inverse-dynamics algorithm. All neural models were implemented in PyTorch and optimized with Adam or PPO updates.

More specifically, the learned components were the autoencoder, sequence models, inverse-dynamics models, behavioral-cloning policies, discriminator, PPO policy, value network, and world-model modules. The non-learned components included frame extraction, grayscale conversion, resizing, normalization, transition construction, action clipping, pseudo-label generation from trained models, and rollout evaluation.

Finally, we anticipated that expert actions would not always be recoverable from visual transitions alone. In `LunarLanderContinuous-v3`, different thrust com-

binations can sometimes produce similar short-term visual changes. This appeared in the experiments: the inverse-dynamics model outperformed a zero-action baseline, but one action dimension was substantially harder to predict than the other. We also encountered a gap between reconstruction or prediction losses and actual policy performance: low autoencoder or latent prediction loss did not automatically imply high environment return.

3. Experiments and Results

3.1. Discriminator-Based Latent State Transitions

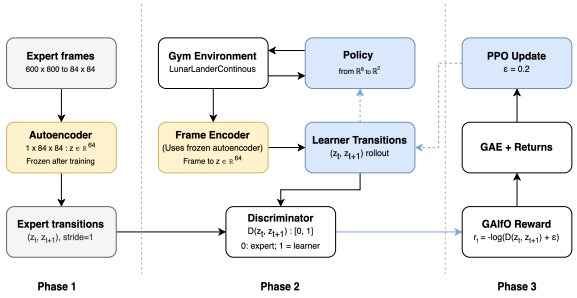

Figure 1. *GAIfO* pipeline. Expert and learner frame transitions are encoded into latent space, compared by a discriminator, and converted into rewards for PPO updates.

Baseline Learner. Student transitions are collected over 2048 steps. At each step t, the current frame is encoded as z_t with the same encoder used for teacher transitions. At state s_t, we sample action a_t from an actor-critic policy. Using a_t as input, we compute the next state s_{t+1}, allowing us to encode the next frame z_{t+1}.

Discriminator. The discriminator (D_w) attempts to distinguish between student and teacher transitions with binary cross-entropy loss D_L in Eq. (1) [6]. The discriminator architecture is a simple 3-layer MLP which takes (z_t, z_{t+1}) latent vectors as input, and outputs a probability between teacher and student transitions where $(D_w(z_t, z_{t+1}) \in (0, 1))$.

$$D_L = -\mathbb{E}_{(z_t, z_{t+1}) \sim teacher}[\log(1 - D_w(z_t, z_{t+1}))] \\ -\mathbb{E}_{(z_t, z_{t+1}) \sim student}[\log(D_w(z_t, z_{t+1}))] \quad (1)$$

Student Model Rewards. In addition, we use Proximal Policy Optimization (PPO) [11] instead of Trust Region Policy Optimization (TRPO), as suggested in the original *GAIfO* implementation. PPO provided a simpler implementation with better handling of stochasticity in the continuous state and action space of `LunarLanderContinuous-v3`. Several past implementations [7, 17] provided a good set of baseline hyperparameters.

Reward calculation in Eq. (2) is based on the original Generative Adversarial Network [3] generator. The reward here replaces the typical environment reward entirely.

$$r_t = -\log(D_w(z_t, z_{t+1}) + \epsilon) \quad (2)$$

Optimizing Policy Updates. Generalized Advantage Estimation (GAE) [10] helps smooth this non-stationary nature of PPO rewards by averaging over multiple timesteps. Existing research [17] provided an implementation that was adapted to our use case. For a successful landing, we lean more towards the next state, therefore $\lambda = 0.95$ gave us reasonable results.

Computed GAE advantages and values in Eq. (3) are then used for PPO updates in its clipped surrogate objective. Clipping prevents the policy update from changing drastically in one update.

$$R_t = \hat{A}_t + V(s_t) \quad (3)$$

Gradient Flow. Backpropagation happens through the PPO update step via two different paths. The first path optimizes the policy loss, and the other optimizes the value loss. However, these two gradients do not interact with each other directly. These gradients update the policy's log probabilities, allowing this information to enter the discriminator's computational graph indirectly.

Evaluation.

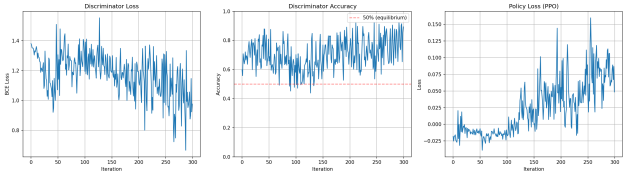

Figure 2. *Left*: Discriminator BCE loss drops in the beginning and rises again as the policy improves. *Center*: Indicates if the Discriminator is able to tell teacher and student apart. *Right*: PPO Policy Loss

Discriminator and Policy Loss. Figure 2 shows D_L from Eq. (1) over 300 iterations for 5 epochs at a learning rate of 10^{-4}.

Discriminator accuracy hovering above 50% indicates that the policy updates can be improved. The policy loss oscillates due to the changing landing surface with each discriminator update.

Qualitative Analysis. Figure 3 shows a successful landing recreated by the Student model. The Teacher model often lands closer to one pole. This behavior is also imitated by the Student model landing closer to one pole. Additionally, visual evaluation of Student's final frames indicated that the side thruster was continuously firing even after landing. This was due to lack of Teacher frames post-landing. This was mitigated by introducing 2 extra seconds of footage post-landing from the expert.

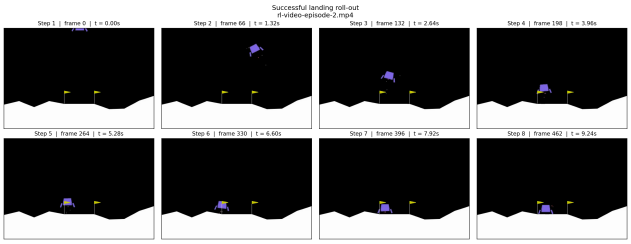

Figure 3. Selected few frames from a successful landing

3.2. Supervised Inverse-Dynamics and Behavioral Cloning

This experiment evaluates a supervised baseline composed of inverse dynamics followed by latent behavioral cloning. We used the aligned teacher transition files to form 6103 expert samples of the form $(frame_t, frame_{t+1}, state_t, a_t, state_{t+1}, r_t, done_t)$, giving us synchronized visual transitions and ground-truth actions for controlled evaluation.

After encoding each frame pair as (z_t, z_{t+1}), we trained an inverse dynamics model to predict the teacher action:

$$\hat{a}_t = f_{\text{IDM}}\big([z_t, z_{t+1}, z_{t+1} - z_t]\big).$$

The model was a multilayer perceptron trained with mean-squared error on normalized actions. We compared it against a zero-action baseline, since `LunarLanderContinuous-v3` actions are centered around zero and a trivial predictor can sometimes appear deceptively strong. The inverse model achieved MSE $= 0.109$ and MAE $= 0.227$, compared with zero-baseline MSE $= 0.274$ and MAE $= 0.439$. This shows that latent visual transitions contain recoverable information about the expert action.

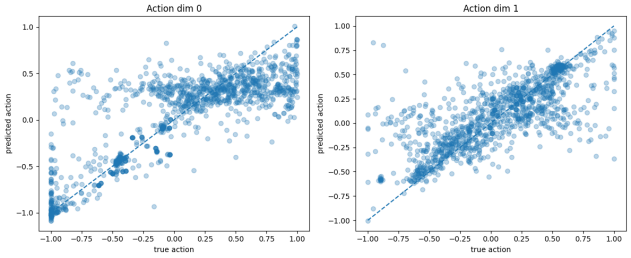

Figure 4. Inverse dynamics predictions against ground-truth teacher actions. The model recovers useful action structure from latent visual transitions, although the two continuous action dimensions are not equally easy to infer.

The per-action analysis showed that the model predicted the two action dimensions with different accuracy. The first action dimension reached MSE $= 0.129$ and MAE $= 0.252$, while the second reached MSE $= 0.090$ and MAE $= 0.202$. Figure 4 also shows that predictions follow the diagonal trend more clearly in one dimension than the other. This supports one of our expected limitations: short visual transitions do not always uniquely determine the action that caused them, especially when different thrust combinations can produce visually similar immediate changes.

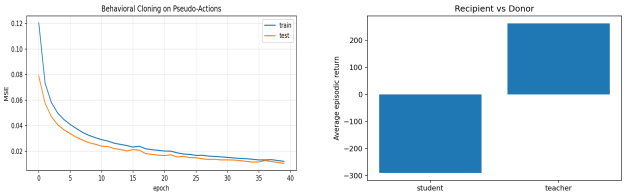

Figure 5. Behavioral cloning from pseudo-actions. Left: supervised loss decreases smoothly, showing that the latent policy fits pseudo-labels generated by the inverse-dynamics model. Right: closed-loop return remains far below the teacher, showing that fitting pseudo-actions does not directly imply stable control.

After training the inverse dynamics model, we used it to pseudo-label the expert visual transitions. This converted the observation-based imitation problem into a behavioral cloning dataset $z_t \to \hat{a}_t$.

We trained a latent policy π_{BC} with two hidden layers and a final `tanh` output so that predicted actions remained in the valid range $[-1, 1]$:

$$\pi_{\text{BC}}(z_t) \approx \hat{a}_t.$$

The behavioral cloning model was trained with mean-squared error against the pseudo-actions. As shown in Figure 5, the training and test losses decreased smoothly, with the best test MSE reaching approximately 0.013. This indicates that the student policy successfully learned the pseudo-action distribution produced by the inverse dynamics model.

However, fitting pseudo-actions did not translate into strong closed-loop control. We evaluated the learned behavioral cloning policy directly in `LunarLanderContinuous-v3` over 10 episodes and compared it against the teacher. The teacher achieved an average return of 260.85 ± 22.94, while the behavioral cloning student achieved -176.98 ± 119.21. Figure 5 summarizes this gap. Qualitatively, saved rollout videos showed that the student could produce nontrivial actions but failed to consistently stabilize and land.

Overall, this baseline partially succeeded. The inverse dynamics model outperformed a trivial zero-action predictor, and the behavioral cloning policy fit the inferred pseudo-actions well. The failure occurred at the policy level: small inverse-dynamics errors and behavioral-cloning errors compounded during rollout, probably causing the student to visit states outside the expert distribution. This result was important because it showed that good supervised losses are not sufficient evidence of successful video-

to-policy learning. It also motivated the discriminator-based latent transition approach, where the learner is evaluated through its own generated transitions rather than only through offline pseudo-label fitting.

3.3. Behavioral Cloning from Observation (BCO)

This experiment extends the supervised IDM baseline from Section 3.2 by removing the dependency on expert action labels inspired by [13], with the main difference between the paper and our approaches being that we operate on a frame-level while the authors directly operate on the environment states.

Instead of training the inverse dynamics model (IDM) on labeled expert transitions, we train it on trajectories generated by a random policy, which allows us to know the actions that were taken since we came up with them and recorded the resulting trajectories. The trained IDM is then used to generate pseudo-labels for the given expert observation trajectories that we do not have actions for, and then we train a Behavioral Clone (BC) policy model on the inferred actions and compare it to a random policy as baseline and the teacher as an upper-bound.

We collected 500 expert episodes (164k frames) and retrained the autoencoder with early stopping and additional landing frames to improve the latent quality. The IDM is an MLP trained on a pair mapping $(\mathbf{z}_t^{(k,s)}, \mathbf{z}_{t+1}^{(k,s)}) \rightarrow \hat{a}_t$ with k latents stacked at stride s gathered from random rollout data while the BC policy takes a single window $\mathbf{z}_t^{(k,s)} \rightarrow \hat{a}_t$. As single frames cannot encode the velocity, the models are unable to distinguish visually identical states that require different actions. This is addressed by frame stacking: Concatenating latents over the temporal dimensions as input to the IDM and the policy.

We compared IDM performance when trained on random versus expert data and observed similar performance, with both around 0.2 MSE. This suggests that the model captures environment physics instead of memorizing a policy.

Method	Config	Reward	Test MSE
Teacher (TD3)	–	261 ± 26	–
Random policy	–	-234 ± 20	–
BCO	1 frame	-284 ± 8	0.122
BCO	4f stride 16	-159 ± 15	0.108
BCO	4f stride 32	-130 ± 5	0.102
BCO	4f stride 64	-128 ± 10	0.083
BCO	8f stride 16	-134 ± 1	0.099

Table 1. BCO with frame stacking compared to baselines and upper-bound. Wider strides improve average reward up until reaching a plateau around -130.

We ran various configurations three times each and report average episodic reward and test MSE as shown in table 1. Frame stacking with stride 32 and 64 achieved approximately -129, above random (-234) but far below the teacher (261). The test MSE continued to decrease with wider windows, but the reward plateaued, which indicates that the network is able to fit the training data but could not generalize to the closed-loop control. Its core limitation is operating at the frame-level: Single frames do not contain dynamics, and while frame stacking is able to partially recover motion through the temporal dimension, the signal is not strong enough for reliable control. A representation that is aware of dynamics, learned jointly with an action objective instead of purely from reconstruction as our current autoencoder, would help here to bridge this gap.

3.4. World Model + Machine Forward

This direction tests the most action-free version of our video-to-policy pipeline. Instead of training directly on expert actions or discriminator rewards, the model learns latent dynamics from unlabeled expert video and uses a machine-forward search step to choose actions at runtime. This direction is related to model-based reinforcement learning and world-model methods, where compact latent dynamics are learned and then used for planning or control [4, 5, 12]. It also connects to inverse- and forward-dynamics representation learning, where actions and state transitions are inferred through predictive objectives [9].

Let o_t denote the rendered frame at time t. The full world-model pipeline can be written compactly as:

$$
\begin{aligned}
z_t &= \mathrm{AE}(o_t), \quad z_t \in \mathbb{R}^{64} \\
h_t, \bar{z}_{t+1} &= \mathrm{LSTM}(z_1, \ldots, z_t) \\
m_t &= \mathrm{AE}(\mathrm{Overlay}(o_1, \ldots, o_t)) \\
\tilde{a}_t &= \mathrm{IDM}(z_t, \bar{z}_{t+1}, h_t, m_t) \\
\Delta\hat{z}_{t+1} &= \mathrm{FDM}(z_t, \tilde{a}_t) \\
a_t^* &= \arg\min_{a \sim \mathcal{U}(-1,1)} \|\mathrm{MF}(z_{t-k:t}, a) - \Delta\hat{z}_{t+1}\|^2
\end{aligned}
$$

The autoencoder maps each frame into a latent vector, the LSTM predicts the next latent state, and the overlay embedding provides visual memory. The IDM infers a latent action, the FDM predicts the desired latent transition, and the Machine Forward model selects the real action whose predicted transition best matches that desired transition.

Before training the downstream components, the autoencoder and LSTM were trained as representation and prediction backbones. The autoencoder reached train/test losses of 0.008/0.014, while the LSTM next-latent prediction loss decreased from 0.494 to 0.033.

Video Learner. The IDM and FDM were trained jointly for 30 epochs with no access to ground-truth actions. As shown in Figure 6, the latent prediction loss dropped from 0.060 to 0.0009, reaching 7× below the LSTM baseline of 0.0067. Cosine similarity between predicted and true latent

transitions improved from 0.024 to 0.37. Qualitatively, the video learner also outperformed the LSTM baseline, with loss 0.0007 compared with 0.0047. However, latent action magnitude saturated near 0.56 while latent action std stabilized around 0.15, suggesting partial action collapse.

Machine Forward and Runtime Control. The Machine Forward model was trained on 100 random-action probe episodes for 30 epochs. MSE converged from $5.96\text{e}{-4}$ to $8.7\text{e}{-5}$ on train and $9.5\text{e}{-5}$ on test. The zero-action baseline MSE was $3.13\text{e}{-4}$, giving a ratio of $3.3\times$, showing the model learned action-conditioned dynamics rather than predicting zero change. At runtime, however, the agent achieved only -269.7 reward over 107 steps. Search loss varied smoothly between $3\text{e}{-4}$ and $1.5\text{e}{-3}$, but both machine and adapter actions collapsed to near-zero across all steps with action std of approximately 0.006, so the agent fell without correction.

Failure Analysis. The action collapse traces back to the IDM training objective. Since the only supervision signal is $L_z = \|\hat{z}_{t+1} - \bar{z}_{t+1}\|^2$, the IDM can minimize this loss by producing a single average latent action that fits the mean transition across the dataset. The LSTM predictions $\bar{z}_{t+1}$ are smooth and consistent, so the IDM converges to a fixed latent vector that satisfies the loss globally. The Machine Forward model then maps this fixed desired transition to a near-zero action, since small actions produce the smallest average latent changes. Without a diversity penalty or any action supervision, there is no signal to break this degeneracy.

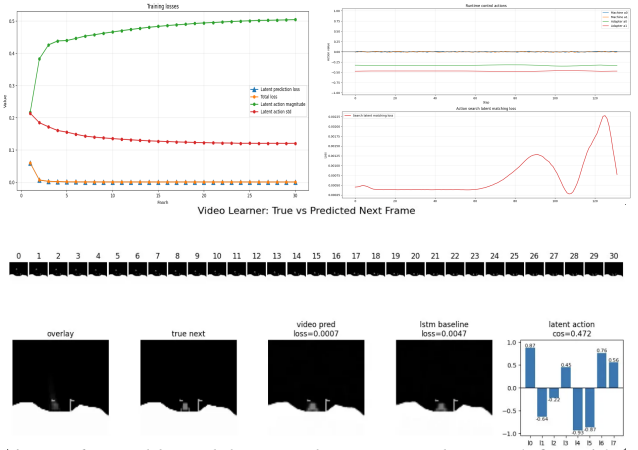

Figure 6. *World Model + Machine Forward.* Top left: Video Learner training curves. Top right: runtime rollout with near-zero action collapse. Bottom: qualitative frame prediction.

3.5. Architecture Generalization Diagnostic

To compare architectures independently of task reward, we ran a synthetic latent-signal propagation probe, shown in Fig. 7. Each model received the same smooth latent sequence. We measured sensitivity to small perturbations, temporal-order usage under sequence reversal, output diversity across the batch, and normalized parameter count. The generalization ratio summarizes these four diagnostic quantities.

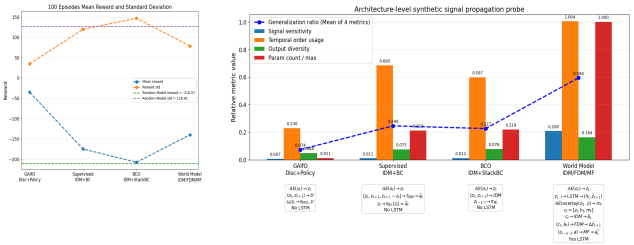

Figure 7. Synthetic architecture-level probe comparing task reward, signal sensitivity, temporal-order usage, output diversity, and normalized parameter count across the four model families.

This probe evaluates architectural generality rather than immediate policy success. GAIfO remains compact and effective for the specific control loop, while IDM and BCO add limited transition-level temporal structure. The world-model pipeline scores highest because it combines latent state, LSTM history, overlay memory, latent actions, forward dynamics, and machine-forward search. This supports the main motivation for the world-model direction: it is the most general and reusable architecture in principle, even though its current closed-loop reward remains limited by action grounding.

4. Future Work

Our results suggest a clear tradeoff. The discriminator-based method produced the strongest closed-loop behavior, but remains tied to environment interaction, discriminator rewards, and policy optimization. The more video-based methods are more general in principle, and the architecture probe supports this most strongly for the world-model pipeline. However, higher capacity and stronger temporal-order usage are not sufficient for stable control unless the learned representation is grounded in controllable actions.

The main next step is therefore to improve action grounding. In the World Model + Machine Forward pipeline, the model learned predictive latent dynamics and showed the strongest architectural generalization signal, but the inferred action signal collapsed toward low-diversity actions. Future versions should add diversity, entropy, or controllability objectives so that different latent actions produce distinguishable future transitions. Training the IDM and FDM on longer episode segments could also help the model learn how desired transitions change across different phases of landing.

Future work should also improve temporal representation learning with longer histories, recurrent or transformer video encoders, and contrastive predictive objectives. However, temporal modeling alone is not enough: it must be tied to action variables that remain useful during closed-loop rollout.

5. Work Division

Team Member	Contributions
Adrian Kazi	**World Model + Machine Forward** 3.4. Worked in the `ak-dev` branch. Trained the expert teacher policy and collected 500 demonstration episodes. Implemented preprocessing, keyframe filtering, the autoencoder, LSTM world model, sequential dataset pipeline, IDM/FDM, Machine Forward model, and runtime random-shooting control loop. Designed Architecture Generalization Diagnostic experiment.
Matteo Bigliardi	**Supervised Inverse-Dynamics and Behavioral Cloning** 3.2. Worked in the `mb-dev` branch on aligned teacher transition export to `.npz` files. Implemented inverse dynamics from latent transitions, pseudo-action generation, latent behavioral cloning, and helped organize the report.
Nuwan Werellagama	**Discriminator-Based Latent State Transitions** 3.1. Built the discriminator-based pipeline in the `nw-dev` branch, incorporating a policy-based reward model. Trained the model and tuned hyperparameters to achieve a successful landing.
Sebastian Nickels	**Behavioral Cloning from Observation (BCO)** 3.3. Worked in the `sn-dev` branch. Ran autoencoder and next-latent prediction ablations, trained the IDM with frame stacking on random-policy data, pseudo-labeled expert demonstrations, and trained the BC policy on those pseudo-labels.

Table 2. Summary of team member contributions.

References

[1] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. In *NeurIPS*, 2022. 2

[2] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *arXiv preprint arXiv:1803.09010*, 2018. 2

[3] Ian J. Goodfellow. Generative adversarial networks. *arXiv preprint arXiv:1406.2661*, 2014. 3

[4] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. 5

[5] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019. 5

[6] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *NeurIPS*, 2016. 3

[7] Ken K. Hong. Lunar lander continuous control with proximal policy optimization and generalized advantage estimation. GitHub repository, 2025. 3

[8] Adrian Kazi, Matteo Bigliardi, Nuwan Werellagama, and Sebastian Nickels. video-to-policy. GitHub repository, 2026. 2

[9] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, 2017. 5

[10] John Schulman. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2018. 3

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 3

[12] Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991. 5

[13] Faraz Torabi, Garrett Warnell, and Peter Stone. Behavioral cloning from observation. In *IJCAI*, 2018. 2, 5

[14] Faraz Torabi, Garrett Warnell, and Peter Stone. Generative adversarial imitation from observation. *arXiv preprint arXiv:1807.06158*, 2018. 2

[15] Faraz Torabi, Garrett Warnell, and Peter Stone. Recent advances in imitation learning from observation. *arXiv preprint arXiv:1905.13566*, 2019. 2

[16] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. 2

[17] Wenyuan Zhao. A pytorch implementation of proximal policy optimization with gae. GitHub repository, 2020. 3